

Pandas Patterns You'll See This Course — Session 1

Machine Learning in Geoscience — pre-session reference · read before Session 1

Same idea as always: this is a recognition guide, not a tutorial. Session 1 is mostly about *loading, cleaning and looking at* raw station data before any modeling happens — so the patterns below are about reading a messy real file and exploring a time series, not about ML itself. Skim it once so these look familiar when they appear in the notebook.

1. Reading and parsing raw data

`pd.read_excel(path)`

Loads an Excel file into a DataFrame — one row per record, one column per field, exactly as it looks in the spreadsheet.

`pd.to_datetime(col, errors='coerce')`

Converts a text column into real datetime objects, so pandas can do time-aware things with it later (sorting, slicing by date, resampling). Timestamps stored as text like "2025-03-10 23:50:10" are not usable as dates until this runs. `errors='coerce'` turns anything unparsable into `NaT` instead of crashing.

`df.sort_values('datetime')`

Puts rows in chronological order. Real exports are often saved newest-first — if you build a lag feature before sorting, `shift(1)` silently grabs the *next* record instead of the previous one.

2. Reshaping the time axis

`df.set_index('datetime')`

Makes the timestamp the row label instead of a normal column. Almost every time-aware pandas operation (resampling, date-slicing, reindexing) expects a datetime index.

`df.reindex(full_hourly_range)`

Forces the data onto a perfectly regular grid (one row per hour, no gaps in the index itself). Missing hours become visible as `NaN` rows instead of silently disappearing — critical, because a silent gap makes "one hour earlier" mean something different depending on where you are in the record.

`df.resample('MS').mean()`

Groups rows into fixed time buckets (here, calendar months — `'MS'` = month start) and aggregates each bucket. This is how the 10-minute logger data becomes an hourly series, and how an hourly series becomes a monthly summary.

```
hourly = raw.resample('h').mean()      # 10-min records -> hourly means
monthly = hourly.resample('MS').mean() # hourly -> monthly means
```

`df.loc['2024-06-01': '2024-06-30']`

Date-string slicing on a datetime index — selects every row between two dates, inclusive. Only works once the index is a real datetime index (see above).

Before Session 1 — a 10-minute self-check

- Make a small toy DataFrame with a text date column and practice: parse it with `pd.to_datetime`, sort it, set it as the index, then slice a date range with `.loc[]`.
- Try `.groupby(df.index.hour).mean()` on the same toy data and see what shape comes out.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 1 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.

3. Grouping and correlation

`df.groupby(df.index.hour).agg(['mean', 'std'])`

`df.index.hour` extracts just the hour-of-day (0-23) from every timestamp. Grouping by it collapses over a year of data into "what's typical at 3am vs 3pm" — this is how a diurnal cycle is measured.

`series.autocorr(lag=k)`

Correlation of a series with a delayed copy of itself. High autocorrelation at lag 1 means "next hour looks a lot like this hour" — this is *why* shuffled train/test splits leak (Session 2): neighboring hours are far from independent.

`df.corr()`

Pairwise correlation between all numeric columns, at the same timestamp — used here to see which raw variables move together (e.g. pressure and wind speed).

4. Describing a distribution

`series.describe(percentiles=[...])`

Count, mean, std, min, max, plus whichever percentiles you ask for — the fastest way to sanity-check a column before trusting it.

`series.skew()`

A single number describing asymmetry. Positive = a long tail of large values (e.g. wind speed: mostly calm, with rare storms pulling the tail right).

Always print `min() / max()` before trusting `mean()` — a sentinel value like `-99999` for "sensor failed" will silently wreck every summary statistic until it's replaced with `NaN`.

5. Marking bad values as missing

`df.loc[df[c] < lo, c] = np.nan`

Boolean masking: select rows failing a physical-plausibility check (e.g. negative wind speed) and overwrite just those cells with `NaN`, leaving everything else untouched.

WRONG — sentinel treated as a real reading
`df['humidity'].mean()`

RIGHT — mark impossible values first, then summarize
`df.loc[df['humidity'] < 0, 'humidity'] = np.nan`
`df['humidity'].mean()`